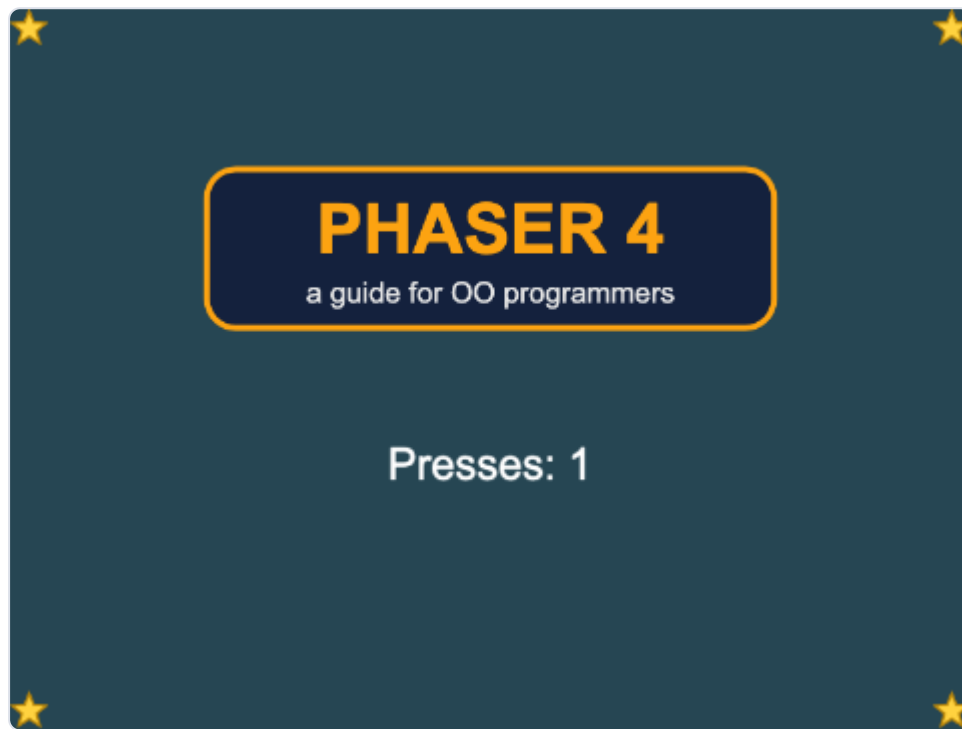




Introduction: your first Phaser 4 game

Phaser is a library for making 2D games that run in a web browser. In this chapter you set up the tools, look inside a Phaser project, meet just enough TypeScript to read it, and run two small games: one that shows pictures and text and responds to a key, and one with a ball that moves by itself.



What you will learn

- what Phaser is, and what is new in Phaser 4
- how a project is built and run with Deno, and opened in Celbridge
- the TypeScript you need, explained for Java programmers
- the three parts of every Phaser game: a **config**, **scenes**, and **game objects**
- how the screen's coordinates work, and what an **origin** is
- what the **game loop** is, and why speeds are measured in pixels per **second**

The projects

Project	What it shows
<code>ch01_hello_phaser</code>	a config, one scene, a loaded picture, text, and a key that does something
<code>ch01_moving_ball</code>	a class of our own that extends a Phaser game object and moves itself every frame

What is Phaser?

[Phaser](#) is a free, open source game framework for the web. You write the game's rules; Phaser does the rest - it draws everything (fast, using the graphics card), runs the game loop, loads pictures and sounds, reads the keyboard, mouse and touch screen, plays audio, runs the physics, and much more. A Phaser game is a web page, so it runs on anything with a browser: Windows, macOS, Linux, phones, tablets.

Phaser 4 came out in 2025. Almost everything you write is the same as in Phaser 3 - scenes, sprites, physics, input, tweens all work the way they did - but the drawing engine underneath was rewritten, and some effects changed. That matters when you search online: most tutorials and answers were written for Phaser 3. They are nearly always still right, but if something from a tutorial does not compile, check the [Phaser 4 migration notes](#) - a handful of features (such as `setTintFill`, bitmap masks and the old "FX" effects) work differently now.

Note - Phaser itself is written in JavaScript, but it comes with complete TypeScript type definitions. That means your editor, and the build, know the type of every Phaser class, method and parameter, and tell you when you get one wrong - long before you run the game.

The tools

Every project in this guide is built the same way, with **Deno** and nothing else to install.

- **Deno** runs TypeScript directly, checks types, bundles code for the browser, and serves web pages - all built in. The only package a project downloads is Phaser itself (the first build fetches it; after that everything works offline).
- **Celbridge** opens each project folder as a project. Two files in every project matter: `terminal.console` (a terminal with buttons for each command) and `game.webview` (a browser panel pointed at the running game).

Running a project

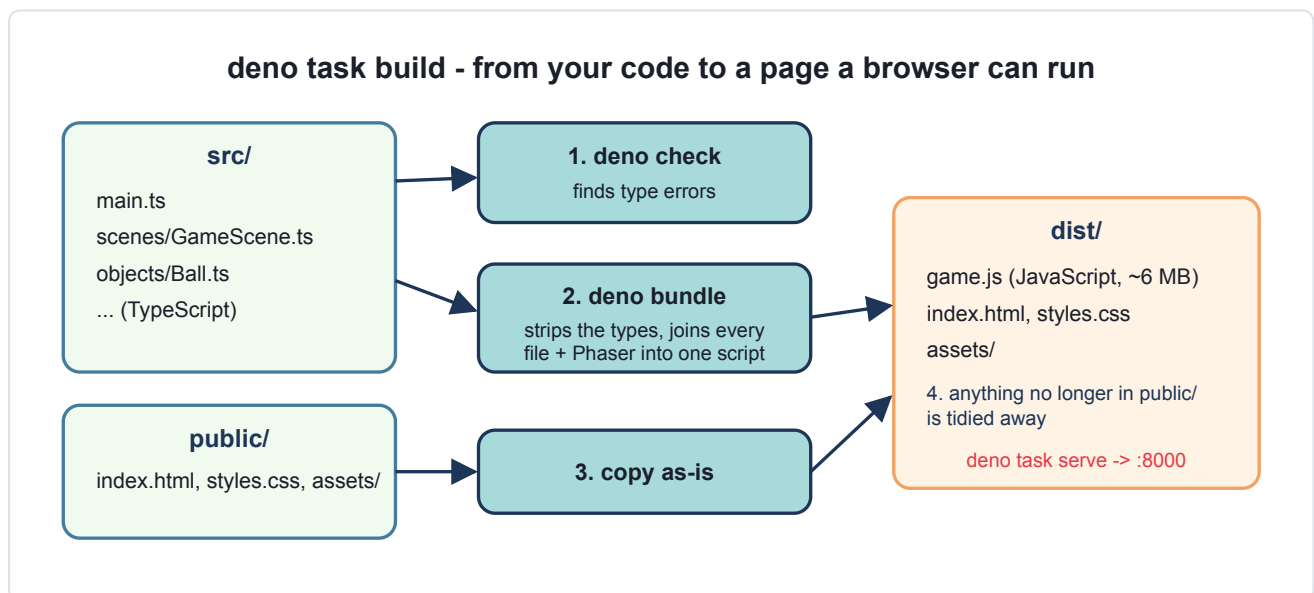
Open the project in Celbridge, open `terminal.console`, and press its buttons in order:

1. **build** runs `deno task build`
2. **serve** runs `deno task serve`, which serves the game at <http://127.0.0.1:8000/>

Then open `game.webview` to play. After you change the code, build again and refresh the game. (`deno task dev` rebuilds every time you save - handy in a second terminal.)

Without Celbridge, the same commands work in any terminal, in the project's folder, and any browser can open <http://127.0.0.1:8000/>.

What "build" does



Browsers cannot run TypeScript, and a game made of many files, plus Phaser, needs joining up. So `build.ts` (which you never need to edit) does four things:

1. **type checks** every `.ts` file - and lists any errors. The game is still built, so you can keep experimenting, but read the errors: they are nearly always a real bug

2. **bundles** `src/main.ts` , every file it imports, and Phaser, into one plain JavaScript file, `dist/game.js`
3. **copies** everything in `public/` (the web page, its CSS, and the game's pictures and sounds) into `dist/`
4. **tidies** `dist/` : anything that is no longer in `public/` (a picture you deleted, say) is removed. `dist/` itself is kept, not deleted and made again, so a page that is showing the game carries on working while you rebuild

Why the game has to be served

Phaser loads pictures and sounds by asking for them over the web, and browsers refuse to do that for a page opened straight from a file (`file:///...`). `deno task serve` runs a tiny web server on your own machine, so the page and its files come from

`http://127.0.0.1:8000/` instead. Nothing is sent anywhere - `127.0.0.1` means "this computer".

A project's files

```
ch01_hello_phaser/
  src/
    main.ts           the game's config - where it all starts
    scenes/
      HelloScene.ts   the scene - what the game does
  public/
    index.html        the web page: a <div id="game"> and a list of the
controls
  styles.css          how the page looks
  assets/images/      the pictures the game loads
  build.ts            the build script - never needs editing
  deno.json           the tasks (build, serve, dev, check) and the one
import: Phaser
  terminal.console    Celbridge's terminal, with a button per task
  game.webview        Celbridge's browser panel
  dist/              the built game - made by the build, never edited
by hand
```

TypeScript for Java programmers

TypeScript is JavaScript with **types** added. If you know Java, most of it will look familiar; the differences are small but they catch everybody out once. Here is what this guide uses.

Variables - `const` for a value that never changes, `let` for one that does. Never `var`.

```
const SPEED = 200;           // type worked out: number
let score = 0;               // type worked out: number
let name: string = "Ada";    // type written out
```

The type goes **after** the name, with a colon. TypeScript works types out for itself wherever it can, so you write them mostly on fields, parameters and return values.

One number type. There is no `int`, `float` or `double` - just `number`. `7 / 2` is `3.5`; use `Math.floor(7 / 2)` for whole-number division.

Equality - always `===` and `!==`. (JavaScript's `==` converts types first, so `"1" == 1` is true. `===` never does.)

Strings - `"double"` or `'single'` quotes, or backticks for a **template string**, which puts values straight into the text:

```
this.message.setText(`Presses: ${this.presses}`);
```

Classes look like Java, with a few differences:

```

export class GameScene extends Phaser.Scene {
    private score: number; // a field: name, then type

    constructor() { // the constructor is
        // called `constructor`
        super("GameScene"); // must call super(...)
        // before using `this`
        this.score = 0; // fields are ALWAYS
        // reached through `this.`
    }

    override update(_time: number, delta: number): void { // return type
        // after the brackets
        this.score = this.score + delta / 1000;
    }
}

```

- fields and methods are reached with `this.` - always. There is no implicit `this` as in Java
- `private`, `protected` and `public` work as in Java (public is the default)
- `override` marks a method that replaces one in the parent class - here, `Phaser.Scene` 's own (empty) `update()` . These projects make it compulsory: leave it out and the build says `This member must have an 'override' modifier` . (And put it on a method that does *not* replace anything, and the build says that too)
- a parameter name starting with `_` (like `_time`) is the usual way to say "I have to accept this, but I do not use it"

Fields set later. A field that gets its value in `create()` , not in the constructor, is declared with a `!` :

```

private message!: Phaser.GameObjects.Text;

```

The `!` says "trust me, this will be set before it is used". Without it, TypeScript complains that the field might never be given a value. (A `!` after an expression, as in `this.input.keyboard!` , means something similar: "this is definitely not `null` ".)

Arrow functions are short functions - like Java's lambdas:

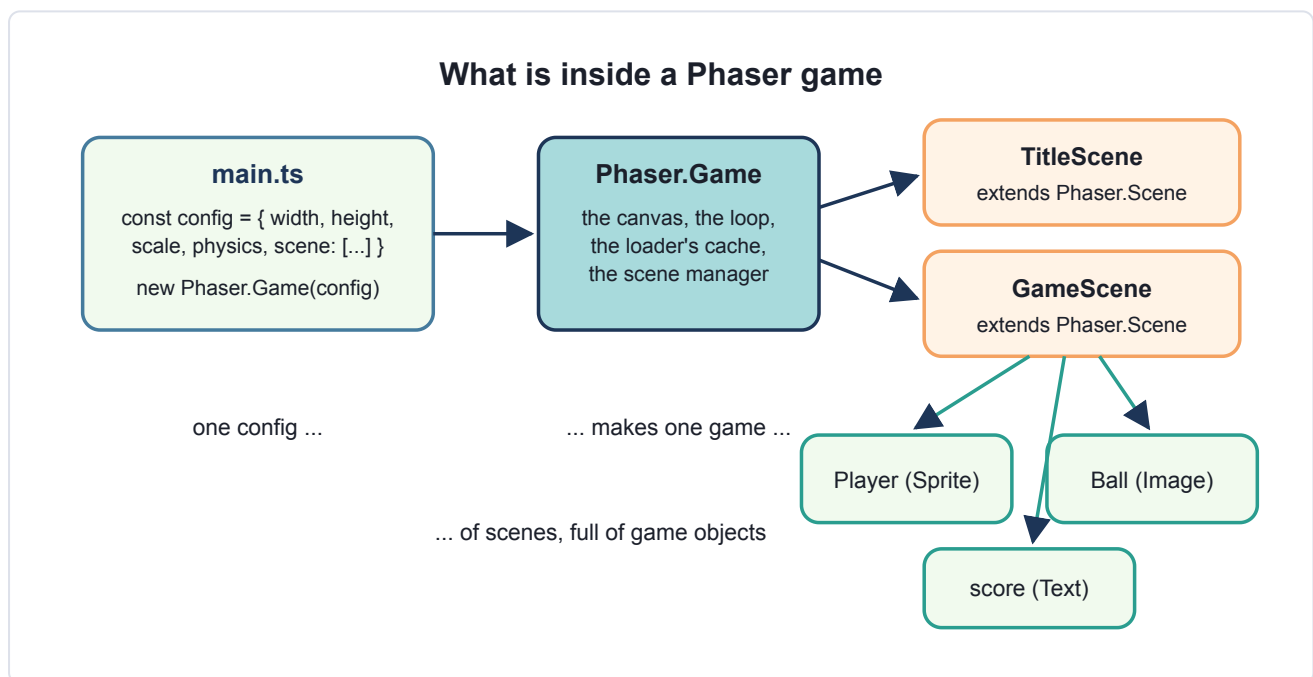
```
this.input.keyboard!.on("keydown-SPACE", () => {
  this.changeColour();
});
```

Inside an arrow function, `this` is still whatever it was outside - here, the scene. That is why the guide uses arrow functions for every callback.

Modules. Each file is a module. `export` lets something leave its file; `import` brings it in. Imports of your own files include the `.ts`:

```
import Phaser from "phaser";
import { HelloScene } from "../scenes/HelloScene.ts";
```

The three parts of a Phaser game



1. The config

`src/main.ts` describes the game and starts it:

`src/main.ts`

```
const config: Phaser.Types.Core.GameConfig = {
  title: "Hello Phaser",
  type: Phaser.AUTO,
  parent: "game",
  backgroundColor: "#1d2433",
  scale: {
    width: 800,
    height: 600,
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH,
  },
  scene: [HelloScene],
};

new Phaser.Game(config);
```

- `type: Phaser.AUTO` - draw with WebGL (the graphics card) if possible, which it nearly always is
- `parent: "game"` - put the game's `<canvas>` inside `<div id="game">` in `index.html`
- `scale` - the game is **always 800 x 600 inside**, whatever size it appears on the page. `Phaser.Scale.FIT` stretches or shrinks it to fit its `<div>`, keeping its shape. Your code only ever deals with 800 x 600
- `scene` - the list of scenes. Phaser makes one of each, and starts the **first** one

The type `Phaser.Types.Core.GameConfig` means TypeScript checks the whole object: misspell `backgroundColor` as `backgroundColour` and the build tells you.

2. Scenes

A **scene** is one screen of the game - a title screen, a level, a game over screen. You write a class that extends `Phaser.Scene`, and fill in the methods Phaser calls for you:

Method	When Phaser calls it	What it is for
<code>constructor()</code>	once, when the game starts	give the scene its key (its name): <code>super("HelloScene")</code>
<code>preload()</code>	when the scene starts	load the pictures and sounds it needs
<code>create()</code>	once everything has loaded	make the game objects
<code>update(time, delta)</code>	every frame, from then on	the game's rules

`src/scenes/HelloScene.ts`

```
preload(): void {
  this.load.image(LOGO_KEY, LOGO_FILE);
  this.load.image(STAR_KEY, STAR_FILE);
}
```

`this.load.image(key, file)` asks the **loader** for a picture. The **key** is the name you use for it from then on; the file path is relative to `index.html`. Chapter 3 is all about loading.

3. Game objects

Everything you see is a **game object**: an image, a sprite, some text, a shape. `this.add` is the scene's **factory** for them - each call makes one, puts it in the scene, and hands it back:

```
const centreX = this.scale.width / 2;

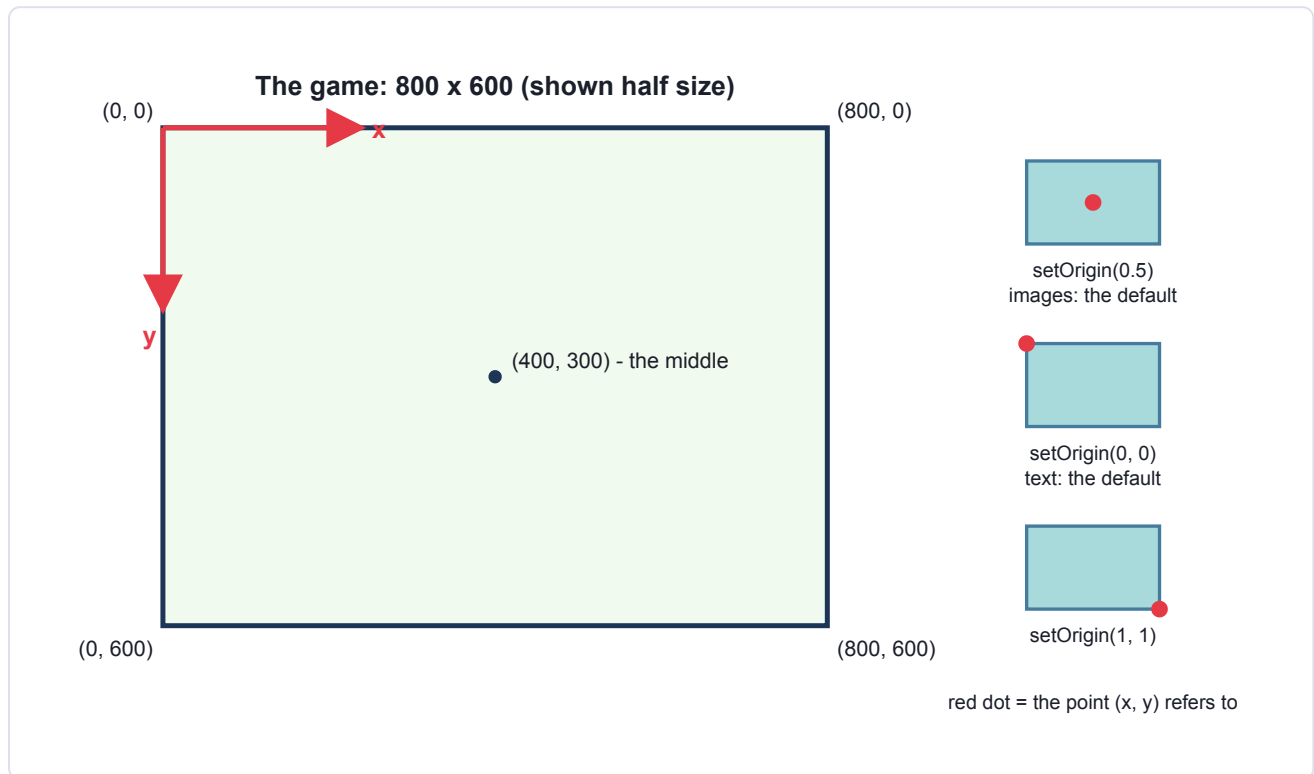
this.add.image(centreX, 200, LOGO_KEY);

this.message = this.add.text(centreX, 380, "Press SPACE", {
  fontFamily: "Arial",
  fontSize: "36px",
  color: "#ffffff",
});
this.message.setOrigin(0.5);
```

Once a game object is in the scene, **Phaser draws it, every frame, until it is removed**. You never draw anything yourself. To change what you see, change the game object - its position, its text, its picture - and the next frame shows the change.

The scene keeps the text in a field (`this.message`) because it changes it later. The logo never changes, so nothing keeps hold of it.

Coordinates and origins



(0, 0) is the **top left** corner. `x` grows to the right and `y` grows **downwards** - the opposite of the graphs you drew at school, and the same as Java's Swing and every other screen.

Every game object has an **origin**: the point on the object that (x, y) refers to. An image's origin starts in its middle - so `this.add.image(400, 300, ...)` centres the picture on the screen. Text starts with its origin at its top left. `setOrigin(x, y)` moves it: 0 is the left or top edge, 1 is the right or bottom edge, 0.5 is half way.

The stars in the Hello Phaser project use origins to sit exactly in each corner, with no sums:

```

this.add.image(0, 0, STAR_KEY).setOrigin(0, 0);
// top left
this.add.image(this.scale.width, 0, STAR_KEY).setOrigin(1, 0);
// top right
this.add.image(0, this.scale.height, STAR_KEY).setOrigin(0, 1);
// bottom left
this.add.image(this.scale.width, this.scale.height, STAR_KEY).setOrigin(1,
1);      // bottom right

```

Most methods that set something return the game object itself, so calls can be **chained** like this. The origin is also the point an object rotates and scales around.

Responding to a key

```

this.input.keyboard!.on("keydown-SPACE", () => {
    this.changeColour();
});

```

`this.input.keyboard` is the scene's keyboard. `.on(event, function)` says "when this happens, run this function" - an **event listener**, like adding an `ActionListener` in Java. The event name is `keydown-` plus the key's name (`SPACE`, `ENTER`, `A`, `LEFT`, ...).

```

private changeColour(): void {
    const colour = Phaser.Utls.Array.GetRandom(COLOURS);
    this.cameras.main.setBackgroundColor(colour);

    this.presses = this.presses + 1;
    this.message.setText(`Presses: ${this.presses}`);
}

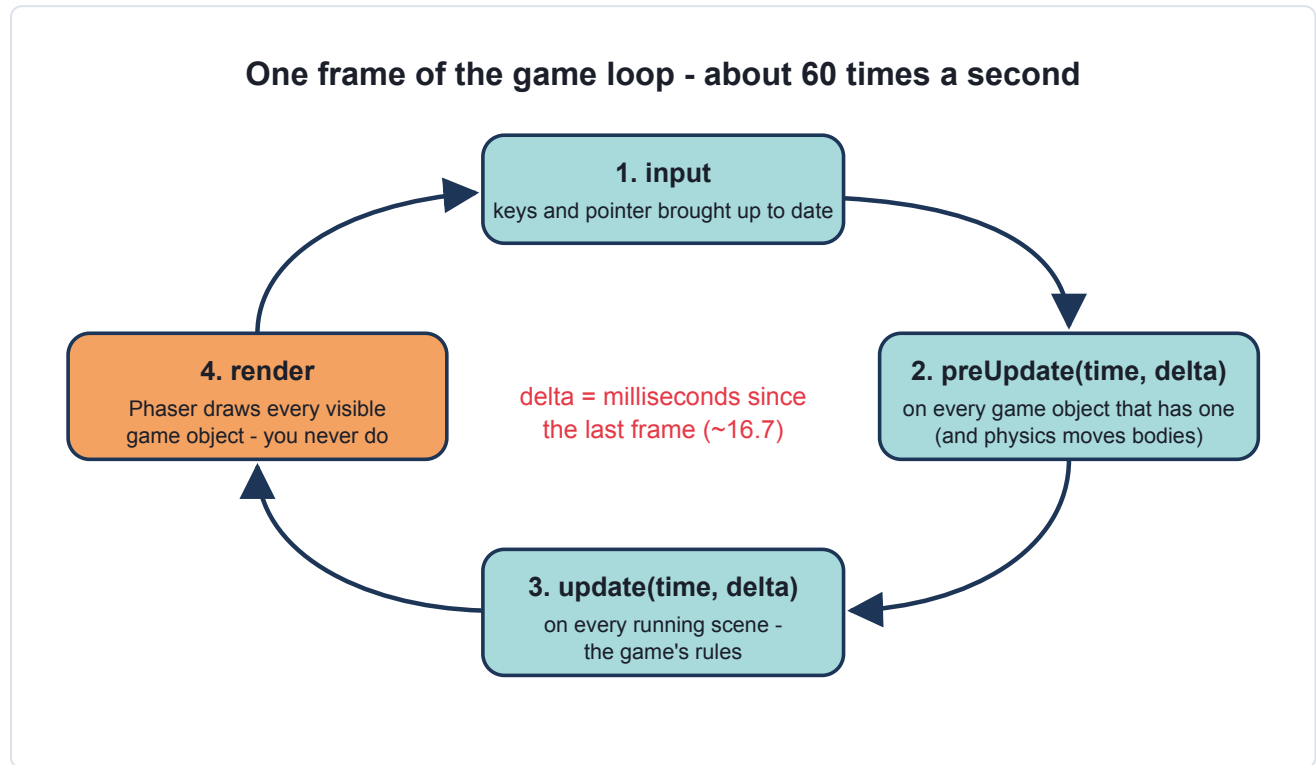
```

Every scene has a **camera** - `this.cameras.main` - which is what looks at the scene and shows it on screen. Its background colour is painted behind everything else.

Try it - build and serve `ch01_hello_phaser`, click on the game (so it has the keyboard), and press SPACE.

The game loop

A game that only changes when a key is pressed is not much of a game. Games change **all the time**: things move, timers count down, enemies think. That happens in the **game loop**.



Phaser runs the loop for you, in time with the screen - usually 60 times a second. Each trip round the loop is a **frame**. In each frame Phaser reads the input, lets every game object and every scene work out what has changed, and then draws everything where it now is.

You join in by writing methods that Phaser calls every frame:

- `update(time, delta)` in a **scene** - the game's rules
- `preUpdate(time, delta)` in a **game object** of your own - the object looks after itself



A game object that moves itself

`ch01_moving_ball` has a class of its own, `Ball`, which **extends** Phaser's `Image`. A ball is an image - it has a picture, a position and a size, and Phaser draws it - plus the ability to move:

`src/objects/Ball.ts`

```
export class Ball extends Phaser.GameObjects.Image {
  private speedX: number;
  private speedY: number;

  constructor(scene: Phaser.Scene, x: number, y: number, speedX: number,
    speedY: number) {
    super(scene, x, y, BALL_KEY);

    this.speedX = speedX;
    this.speedY = speedY;

    scene.add.existing(this);
  }
}
```

`this.add.image(...)` both makes an image and adds it to the scene. When you make a game object of your own with `new`, you have to add it yourself -

`scene.add.existing(this)` does that. It also notices that `Ball` has a `preUpdate()` method, and signs it up to be called every frame.

The scene makes a ball and forgets about it:

`src/scenes/GameScene.ts`

```
create(): void {  
    new Ball(this, 400, 300, 240, 180);  
    ...  
}
```

Speeds in pixels per second

```
preUpdate(_time: number, delta: number): void {  
    const seconds = delta / 1000;  
  
    this.x = this.x + this.speedX * seconds;  
    this.y = this.y + this.speedY * seconds;  
    ...  
}
```

(No `override` here: `Image` has no `preUpdate()` of its own for this one to replace. A `Sprite` does, and Chapter 4 shows what changes then.)

`delta` is how long the last frame took, in **milliseconds** - about 16.7 at 60 frames a second. Dividing by 1000 gives seconds, and **speed x time = distance**. So `speedX = 240` means 240 pixels every second, on every computer.

Why not just add a few pixels every frame? Because screens differ. Most run at 60 frames a second, but many laptops and gaming monitors run at 120 or 144. "Add 4 pixels a frame" would make the ball more than twice as fast on those screens. Measuring time instead of counting frames makes the game play the same everywhere. (From Chapter 8 onwards, Arcade Physics does this sum for you.)

Bouncing

```
const radius = this.width / 2;
const right = this.scene.scale.width - radius;
const bottom = this.scene.scale.height - radius;

if (this.x < radius || this.x > right) {
  this.speedX = -this.speedX;
  this.x = Phaser.Math.Clamp(this.x, radius, right);
}
```

The ball's `(x, y)` is its middle, so its edge touches the side of the screen when `x` is one radius from it. Turning round is just flipping the sign of the speed. `Clamp` puts the ball back inside the edge, in case it went a little past it during the frame.

`this.scene` - every game object knows which scene it is in, so the ball can ask the scene's scale manager how big the game is.

Where to find out more

- the Phaser site, <https://phaser.io/>, has the API documentation and hundreds of examples
- your editor knows Phaser's types: hover over any Phaser method to see what it takes and returns, or type `this.add.` and look at the list
- when a tutorial written for Phaser 3 does not work, the type checker usually points straight at the line that changed

Summary

- A Phaser game is a **config** (in `main.ts`), a list of **scenes**, and the **game objects** in them. Phaser draws every game object, every frame
- a scene is a class that extends `Phaser.Scene`; Phaser calls its `preload()`, `create()` and `update()` at the right times
- `this.add.image(...)`, `this.add.text(...)` make game objects; `setOrigin(...)` says which point of the object `(x, y)` refers to
- `(0, 0)` is the top left; `y` grows downwards; the game is always 800 x 600 inside

- `this.input.keyboard!.on("keydown-SPACE", () => { ... })` responds to a key
 - the **game loop** calls `preUpdate()` on game objects and `update()` on scenes every frame; `delta` is the length of the last frame, in milliseconds - use it to move things in pixels per second
 - build with `deno task build`, serve with `deno task serve`, play at <http://127.0.0.1:8000/>
-

Challenges

1. **Make it yours** (*ch01_hello_phaser*) - Change the game's title (in the config and in `index.html`), the starting background colour, and the message's size and colour. Add two colours of your own to the `COLOURS` list.
 2. **Corner labels** (*ch01_hello_phaser*) - Next to each corner star, add a small piece of text showing that corner's coordinates - `(0, 0)`, `(800, 0)` and so on - lined up against the edges of the screen. Use origins rather than working out text widths.
 3. **Reset key** (*ch01_hello_phaser*) - Add a second key, R, that sets the press count back to 0, puts the background back to its starting colour, and changes the message back to "Press SPACE".
 4. **More balls** (*ch01_moving_ball*) - Make five balls, each starting in a random place and moving at a random speed in a random direction. Use a loop, and `Phaser.Math.Between(min, max)`. *Hint*: a speed can be negative - that is a direction.
 5. **Bounce counter** (*ch01_moving_ball*) - Count how many times the ball has hit an edge, and show the total on screen. The ball knows when it bounces; the scene owns the text. *Hint*: give `Ball` a way for the scene to ask it how many times it has bounced, and have the scene ask in `update()`.
 6. **Steer the ball** (*ch01_moving_ball*) - Let the arrow keys **push** the ball: each arrow held down adds speed in its direction, a little every frame (so holding it longer makes the ball faster), up to a top speed of 600 pixels a second in each direction. Show the ball's current speed on screen. *Hint*: the scene reads the keys; give `Ball` a public method such as `push(amountX, amountY)` so the scene can ask the ball to speed up - and remember that "a little every frame" needs `delta` too.
-

Next: [Chapter 2 - A three-scene game](#)